ensia The National School of Artificial Intelligence المدرسة الوطنية العليا للذكاء الاصطناعي

Introduction to Blockchain

Lab #4

Developing a Voting Smart Contract with Solidity using Remix

Objective:

In this lab, you will develop a simple voting smart contract using **Solidity** in **Remix** and deploy it to an Ethereum test network. You will interact with the contract using **MetaMask** to cast votes and retrieve the results.

Instructions:

Step 1: Create a MetaMask Wallet

Before you can interact with the Ethereum network, you need a MetaMask wallet. MetaMask is a browser extension that allows you to manage your Ethereum accounts and interact with decentralized applications (DApps).

1. **Install MetaMask Extension**:
    - Visit the MetaMask website: https://metamask.io/download.html.
    - Download the MetaMask extension for your browser (Chrome, Firefox, or Brave).
    - After installation, you will see the MetaMask icon appear in your browser's toolbar.
2. **Create a New Wallet**:
    - Click the MetaMask icon in your browser toolbar.
    - Click on **Get Started**, then select **Create a Wallet**.
    - Read and agree to the privacy policy.
    - **Set a Strong Password**: Create a strong password for your MetaMask wallet. This will be used to unlock your wallet in your browser.
    - **Save Your Secret Recovery Phrase**: MetaMask will generate a **Secret Recovery Phrase** (12-word backup phrase). Write it down and store it

somewhere safe. This is crucial for recovering your wallet if you forget your password or need to restore your account.
    - After confirming your recovery phrase, your MetaMask wallet will be created.
3. **Configure MetaMask**:

    - After creating your wallet, you'll be prompted to set your default network. For development, select one of the Ethereum **Test Networks** (such as **Seploia**).
    - **Switch to Test Networks**: In the MetaMask extension, click on the network dropdown at the top and select either **Sepolia** or any test network of your choice.

Step 2: Get Test Ether on Sepolia

To deploy a smart contract to a test network, you'll need test Ether, which can be obtained from a faucet.

1. **Get Sepolia Test Ether**:

    - Visit the **Sepolia Faucet**: Ethereum Sepolia Faucet
    - Follow the instructions on the website to obtain test Ether for the Goerli test network.

Step 3: Setup Remix IDE

1. **Open Remix IDE**:

    - Go to the Remix IDE at Remix - Ethereum IDE in your browser .
2. **Create a New Solidity File**:

    - In the Remix file explorer, create a new file called `Voting.sol` under the `contracts` folder.

Step 4: Write the Voting Smart Contract in Solidity

1. **Write the Voting Contract**:

    - In `Voting.sol`, write the following code:

```
pragma solidity ^0.8.0;


contract Voting {

    mapping(string => uint) public votesReceived;

    string[] public candidateList;


    constructor(string[] memory candidateNames) {

        candidateList = candidateNames;

    }


    function voteForCandidate(string memory candidate)
public {

        require(validCandidate(candidate), "Candidate
not valid.");

        votesReceived[candidate] += 1;

    }


    function totalVotesFor(string memory candidate)
public view returns (uint) {

        require(validCandidate(candidate), "Candidate
not valid.");

        return votesReceived[candidate];
```

```
    }


    function validCandidate(string memory candidate)
public view returns (bool) {

        for (uint i = 0; i < candidateList.length;
i++) {

            if (keccak256(abi.encodePacked(candidate))
== keccak256(abi.encodePacked(candidateList[i]))) {

                return true;

            }

        }

        return false;

    }

}
```

2. **Explanation**:
    - The `Voting` contract allows users to vote for candidates.
    - `voteForCandidate`: Accepts a candidate's name and increments their vote count.
    - `totalVotesFor`: Returns the total votes for a given candidate.
    - `validCandidate`: Checks if a candidate is part of the list of valid candidates.

Step 5: Compile the Smart Contract

1. **Select the Solidity Compiler**:

- In the Remix IDE, navigate to the "Solidity Compiler" tab (on the left sidebar).
- Make sure the compiler version is set to **0.8.x** (to match the version used in the contract).
- Click on **Compile Voting.sol** to compile the contract.

2. **Check for Errors**:

   - Ensure that the contract compiles successfully without errors. If there are errors, Remix will display them in the "Compile" section.

Step 6: Deploy the Smart Contract to the Ethereum Test Network

1. **Connect MetaMask**:

   - Open MetaMask and connect to a test network like **Sepolia**.
   - Ensure that your MetaMask wallet has some test Ether. You can obtain test Ether from faucets like Ethereum Sepolia Faucet (as described in step 2)

2. **Deploy the Contract**:

   - In the **Deploy & Run Transactions** tab (on the left sidebar of Remix), ensure that:
     - **Environment** is set to **Injected Web3**. This connects Remix to MetaMask.
     - **Account**: Choose an account from your MetaMask wallet.
     - **Gas limit**: Set to **Auto** (or adjust based on your contract's size).
     - **Constructor arguments**: Enter an array of candidates, e.g., `["Mohamed", "Omar", "Ali"]` in the **Deploy** section.
   - Click **Deploy** to deploy the contract to the Ethereum test network.

3. **Confirm Deployment**:

   - MetaMask will ask for your confirmation to deploy the contract. Click **Confirm** and wait for the transaction to be mined.
   - After the deployment is complete, Remix will display the contract's address in the **Deployed Contracts** section.

Step 7: Interact with the Contract Using MetaMask

1. **Voting for a Candidate**:
    - In the **Deployed Contracts** section, find your contract and expand the options.
    - Use the **voteForCandidate** function to cast a vote for a candidate. Enter a candidate's name (e.g., "Alice") and click **transact**. Confirm the transaction in MetaMask.
2. **Check Total Votes for a Candidate**:
    - In the **Deployed Contracts** section, expand the options and find the `totalVotesFor` function.
    - Enter the name of a candidate (e.g., "Alice") and click **call** to see the number of votes for that candidate.

Step 8: Implement a Basic Front-End Using HTML and Web3.js

1. **Create an HTML Interface**:
    - Create an HTML file (e.g., `index.html`) to interact with the deployed contract. This HTML page will allow users to vote for candidates via a simple form.

      html
      Copy

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>Voting DApp</title>

</head>
```

```
<body>

    <h1>Vote for Your Favorite Candidate</h1>

    <input type="text" id="candidate"
placeholder="Enter candidate name">

    <button id="vote-button">Vote</button>

    <h3>Total Votes for Mohamed: <span
id="Mohamed-votes">0</span></h3>

    <h3>Total Votes for Omar: <span
id="Omar-votes">0</span></h3>

    <h3>Total Votes for Ali: <span
id="Ali-votes">0</span></h3>



    <script
src="https://cdn.jsdelivr.net/npm/web3@1.2.9/dist/web3
.min.js"></script>

    <script>

        const web3 = new Web3(window.ethereum);

        let contract;



        window.ethereum.enable().then(async () => {

            const contractAddress =
'YOUR_CONTRACT_ADDRESS';  // Replace with the actual
contract address

            const abi = YOUR_ABI;  // Replace with the
actual ABI of your contract
```

```
            contract = new web3.eth.Contract(abi,
contractAddress);


            const accounts = await
web3.eth.getAccounts();



document.getElementById("vote-button").addEventListene
r('click', async () => {
                const candidate =
document.getElementById("candidate").value;
                await
contract.methods.voteForCandidate(candidate).send({
from: accounts[0] });
                alert('Vote cast for ' + candidate);
            });


            // Update vote counts for each candidate
            const updateVotes = async () => {
                const MohamedVotes = await
contract.methods.totalVotesFor("Mohamed").call();
                const OmarVotes = await
contract.methods.totalVotesFor("Omar").call();
```

```
                const AliVotes = await
contract.methods.totalVotesFor("Ali").call();



document.getElementById("Mohamed-votes").innerText =
MohamedVotes;


document.getElementById("Omar-votes").innerText =
OmarVotes;


document.getElementById("Ali-votes").innerText =
AliVotes;
            };


            setInterval(updateVotes, 1000);  // Update
every second
        });
    </script>
</body>
</html>
```

2. **Explanation**:
    - This simple HTML page allows users to vote for a candidate by typing the name and clicking the **Vote** button.
    - The Web3.js script interacts with the deployed contract and updates the vote counts on the page in real-time.

Step 9: Test Your Application

1. **Open the HTML Page**:
    - Open the `index.html` file in a web browser.
2. **Vote and Check the Results**:
    - Ensure your MetaMask wallet is connected to the test network (e.g., Rinkeby).
    - Vote for a candidate and observe the votes updating in real-time on the page.